RELAZIONE

Sommario

Introduzione 1

Struttura dei dati 1

Struct pedina 1

Scacchiera matrice array bi 1

Diagramma di flusso 2

Strategia (sistema di score) 2

Suddivisione file 3

Organizzazione del lavoro 3

Principali difficoltà incontrate 3

Scegliere la struttura giusta 3

Grafica al centro dello schermo 3

RELAZIONE

In questa relazione si andranno ad evidenziare le modalità con cui è stato strutturato il progetto, l'organizzazione del lavoro di gruppo, ed infine, le principali problematiche riscontrate e soluzioni messe in atto.

Introduzione

miniLaska è una variante del gioco originale Laska.

Rispetto al gioco originale miniLaska prevede le seguenti limitazioni:

- si può mangiare/conquistare una sola volta per mossa;
- le torri possono essere alte al massimo 3 pedine, superato questo limite, la pedina più in basso viene rimossa dalla scacchiera.

Prima di scrivere il codice, è stato preso in considerazione un pool di idee per la realizzazione di tale progetto e ne è stata valutata l'efficacia, selezionando accuratamente le proposte migliori.

Struttura dei dati

In seguito a varie considerazioni si è deciso di utilizzare una struttura che fosse il più funzionale possibile applicando diverse soluzioni integrate fra loro, elencate di seguito.

Struct pedina

Per quanto riguarda i pezzi, quindi le pedine, si è optato per una ADT, una struct opportunamente creata per il mantenimento di alcune informazioni necessarie per il funzionamento del gioco. La seguente struct, rinominata per convenzione ped_t contiene:

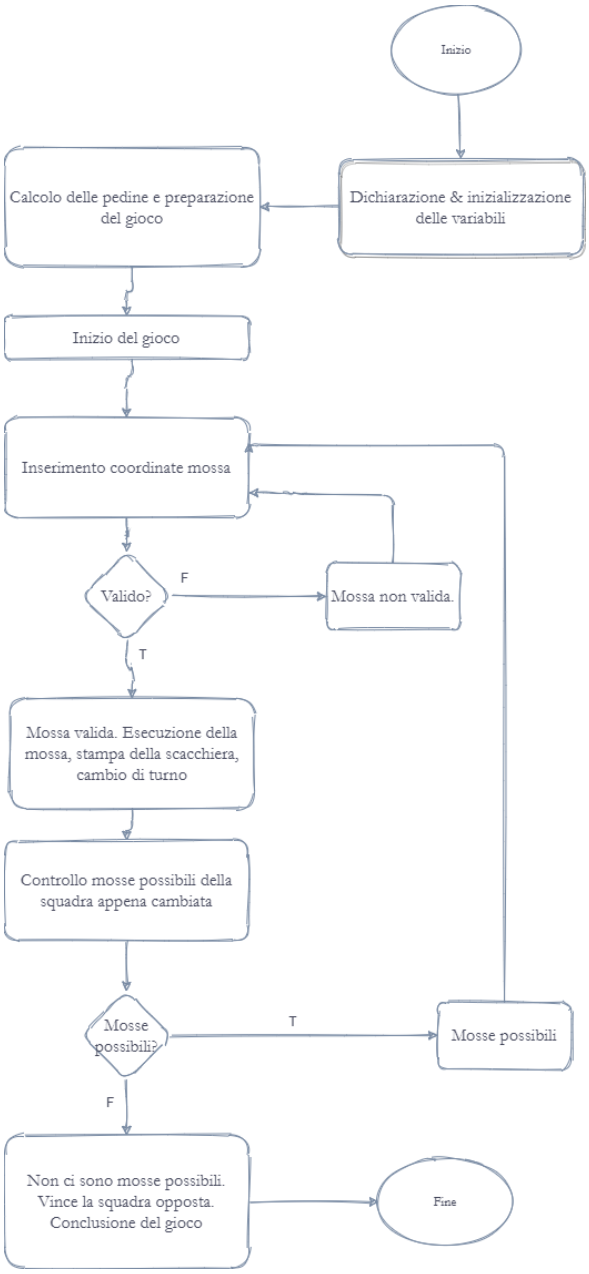
```
coord_t posizione; /*Ovvero le coordinate, opportunamente racchiuse in un nuovo tipo, della pedina.*/
char colore; /* Indica a quale squadra appartiene la pedina */
bool promozione; /*Una variabile flag che specifica se la pedina ha raggiunto la Promozione. */
bool conquistato; /* Utile a sapere se la pedina è stata conquistata */
mossa_con_tipo_t database[8]; /* Database di mosse possibili che la pedina può effettuare(utile alla strategia del
gioco utente vs computer) */
short int mosse_possibili; /*  Numero mosse possibili */
short int mosse_mangia_possibili; /* Mosse per conquistare altre pedine possibili */
short int score;  /*Sistema di punteggi che fa scegliere la mossa migliore quando si gioca con il computer*/
```

Scacchiera matrice array bi

Per la scacchiera, parte fondamentale sulla quale si muoveranno le pedine, è stata utilizzata una matrice, ovvero un array bidimensionale con dimensioni [7][7*3] affinché possa contenere sia tutte le caselle dove si muovono le pedine, sia le aree di gioco non legali, opportunamente segnalate. Infine la seconda dimensione ([7*3]) permetterà di ospitare lo spazio per le sotto caselle di una posizione, ovvero le torri di pedine conquistate, in questa variante del gioco limitate ad altezza 3.

RELAZIONE

Diagramma di flusso



Una volta decise le strutture che fanno parte del progetto, è stata pensata l'evoluzione del gioco, attraverso una serie di step necessari a tale svolgimento. Una volta discusse le varie possibilità, si è deciso di schematizzare il tutto mediante l'utilizzo di un flowchart che rappresenta l'esecuzione del programma.

Strategia (sistema di score)

Oltre alla modalità di gioco standard, ovvero la possibilità per due utenti di giocare uno contro l'altro mediante l'alternanza di mosse in input, è stata prevista la possibilità per un utente di giocare contro il computer. Per far ciò, è stata quindi implementata una strategia che permette al programma di contrastare le mosse dell'utente e persino di vincere se non si sta attenti alle possibili risposte che il computer può eseguire. Il programma è stato ideato mediante un sistema di score, ovvero dei punteggi assegnati a ciascuna mossa in base alla sua pericolosità. L'entità artificiale può così fare delle scelte che possono portare alla sconfitta dell'utente. Il sistema di attribuzione del valore è stato articolato mediante delle funzioni che assegnano differenti punteggi alle numerose mosse possibili. Nella seguente tabella si può osservare il livello di pericolosità della mossa, dato dal relativo score.

Tipo di mossa	Score
Conquista pedina avversaria	5
Spostamento in un'altra casella	3
Coefficiente di posizione (in base alla colonna)	+1 || +2 || +3

Suddivisione file

Tenendo conto dei consigli dei docenti, la scrittura del codice è stata suddivisa in file:
- file header laska.h con tutte le firme di funzioni che servono all'esecuzione del programma, ben documentate tramite l'ausilio di Doxygen;
- file laska.c contenente i sorgenti delle funzioni sviluppate con l'intento di un approccio top-down;
- file main.c che contiene le istruzioni necessarie all'esecuzione del programma con le chiamate alle relative funzioni.

Organizzazione del lavoro

Con l'intento di ottenere dei risultati concreti, è stata necessaria un'organizzazione dettagliata dei compiti per lo svolgimento del lavoro di gruppo. Innanzitutto prendendo il problema principale e scomponendolo in più piccoli sottoproblemi, facilmente trascrivibili in codice tramite delle funzioni. Per garantire il corretto proseguimento dei lavori, sono state organizzate delle riunioni con cadenza bisettimanale, che hanno permesso ai componenti di confrontarsi, aiutarsi e controllare lo stato di avanzamento del progetto.

Principali difficoltà incontrate

Come in qualsiasi lavoro che si rispetti, sono state riscontrate delle avversità, soprattutto all'inizio, mentre si cercava di collaborare, nonostante le differenti scuole di pensiero sulla struttura del progetto.

Scegliere la struttura giusta

Il brainstorming iniziale ha dato alla luce differenti idee in merito alle strutture da impiegare, partendo da idee di array lineari con singola dimensione e finendo con proposte di array frastagliati. Le quali, avendo a loro volta vantaggi e svantaggi hanno richiesto una decisione strategica sulle soluzioni da applicare.
Perciò si è deciso di valutare pro e contro delle varie possibilità. La decisione è ricaduta sulla realizzazione di una struttura mediante un approccio ADT, molto più flessibile alle esigenze di questo gioco.

Grafica al centro dello schermo

Una sfida per noi innovativa è stata l'idea di inserire una grafica all'interno del programma, per far sì che l'utente possa vedere in maniera tangibile lo sviluppo del gioco in sé.
Il tutto per interagire in maniera semplice con il gioco, al fine di raggiungere una visualizzazione esteticamente chiara e leggibile.
È stato ottimale l'utilizzo di funzioni già implementate che permettono di cancellare la videata, permettendo quindi un costante refresh della visualizzazione, che rende il tutto molto più dinamico. Per centrare il tutto, si è optato per una misurazione dei caratteri che occupano lo schermo, ed è bastato calcolare un offset, ovvero la distanza tra le parti dello schermo, per rendere il tutto proporzionato ai fini della corretta visualizzazione.
Al termine di questo progetto è stata apprezzata l'esperienza di lavorare in gruppo alla realizzazione di un progetto per noi innovativo, che ci ha dato molti spunti di miglioramento e la possibilità di mettere in pratica quanto appreso a lezione.